PROJECT IMPLEMENTATION

1. Project Overview:

This project implements a Retrieval-Augmented Generation (RAG) based Customer Support Assistant that:

- Reads a PDF knowledge base (policies / FAQs)
- Splits documents into chunks
- Converts chunks into embeddings
- Stores embeddings in a vector database (ChromaDB / FAISS)
- Retrieves relevant context for user queries
- Uses a simple LLM-style response generator
- Applies LangGraph-style workflow routing
- Supports Human-in-the-Loop (HITL) escalation

2. System Architecture:

The system follows a **modular layered architecture**, where each component has a clearly defined responsibility:

- Data Ingestion Layer → Handles PDF processing
- Embedding Layer → Converts text into vector representations
- Storage Layer → Stores embeddings in vector database
- Retrieval Layer → Fetches relevant context for queries
- Reasoning Layer → Generates answers using retrieved context
- Workflow Layer → Controls execution flow using graph logic
- Escalation Layer → Routes complex queries to human support

This separation ensures **scalability, maintainability, and flexibility**.

3. Technology Stack

- Python 3.10+
- LangChain (for embeddings + loaders)
- Sentence Transformers (MiniLM)
- ChromaDB / FAISS (Vector Storage)
- PyPDFLoader (PDF ingestion)
- Custom LangGraph-style workflow (manual implementation)
- CLI Interface (terminal-based chat system)

4. Data Pipeline

Step 1: PDF Loading

- Input: sample.pdf
- Tool: PyPDFLoader

Step 2: Chunking

- Text split into small chunks (approx 300–500 tokens)
- Ensures better retrieval accuracy

Step 3: Embedding Generation

- Model: sentence-transformers/all-MiniLM-L6-v2
- Converts text → vector embeddings

Step 4: Vector Storage

- Stored in ChromaDB / FAISS index
- Enables similarity search

5. Retrieval System

When a user asks a question:

1. Query is embedded
2. Cosine similarity is computed
3. Top-K relevant chunks are retrieved
4. Context is passed to response generator

6. LLM Response Logic

Instead of raw LLM hallucination:

- Only retrieved context is used
- Answer is extracted from closest matching chunks
- No external knowledge allowed

7. Workflow (LangGraph Simulation)

Nodes implemented:

1. INPUT NODE

- Receives user query

2. INTENT NODE

- Classifies query type:
    - refund
    - shipping
    - cancel
    - payment
    - general
    - escalate

3. RETRIEVAL NODE

- Fetches relevant chunks from vector DB

4. GENERATION NODE

- Builds final response using retrieved context

5. HITL NODE

- Triggered when:
    - Low confidence
    - Escalation intent
    - Complex query

8. Human-in-the-Loop (HITL)

Trigger conditions:

- Query confidence is low
- Query contains escalation keywords (angry, agent, human)
- No relevant document found

Action:

- Route query to human agent interface
- Prevents wrong AI answers

9. Example Output

User Query:

How long does refund take?

System Flow:

- Intent: refund
- Retrieved chunk: refund policy section
- Context matched

Final Answer:

Refunds are processed within 5–7 business days.

10. Error Handling Implemented

- PDF corruption handling
- Empty retrieval fallback
- No context fallback response
- API failure handling
- Invalid query handling

11. Features Achieved

✓ RAG pipeline working
✓ PDF → chunk → embedding → retrieval
✓ Vector DB search working
✓ Intent classification working
✓ CLI chatbot interface working
✓ HITL escalation system implemented
✓ LangGraph-style workflow simulation

12. Final Outcome

This system demonstrates:

- Real-world **AI Support System architecture**
- Production-style **RAG pipeline**
- **Decision-based workflow routing**
- **Hybrid AI + Human support model**